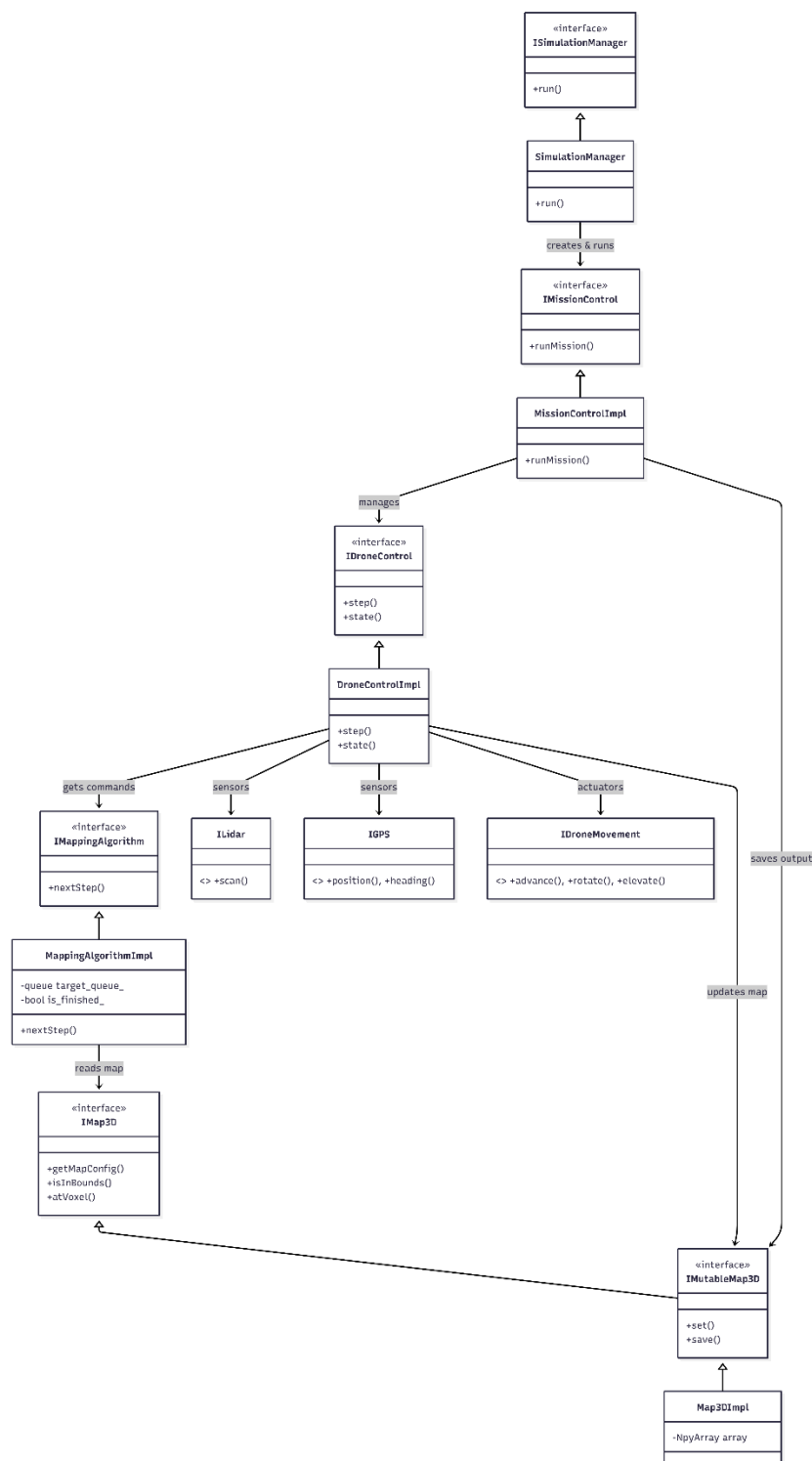


High-Level Design (HLD) - Drone Mapper

1. UML Class Diagram

The following diagram illustrates the system's architecture. The system is designed with a strong emphasis on Separation of Concerns, ensuring each component is responsible for a well-defined domain.



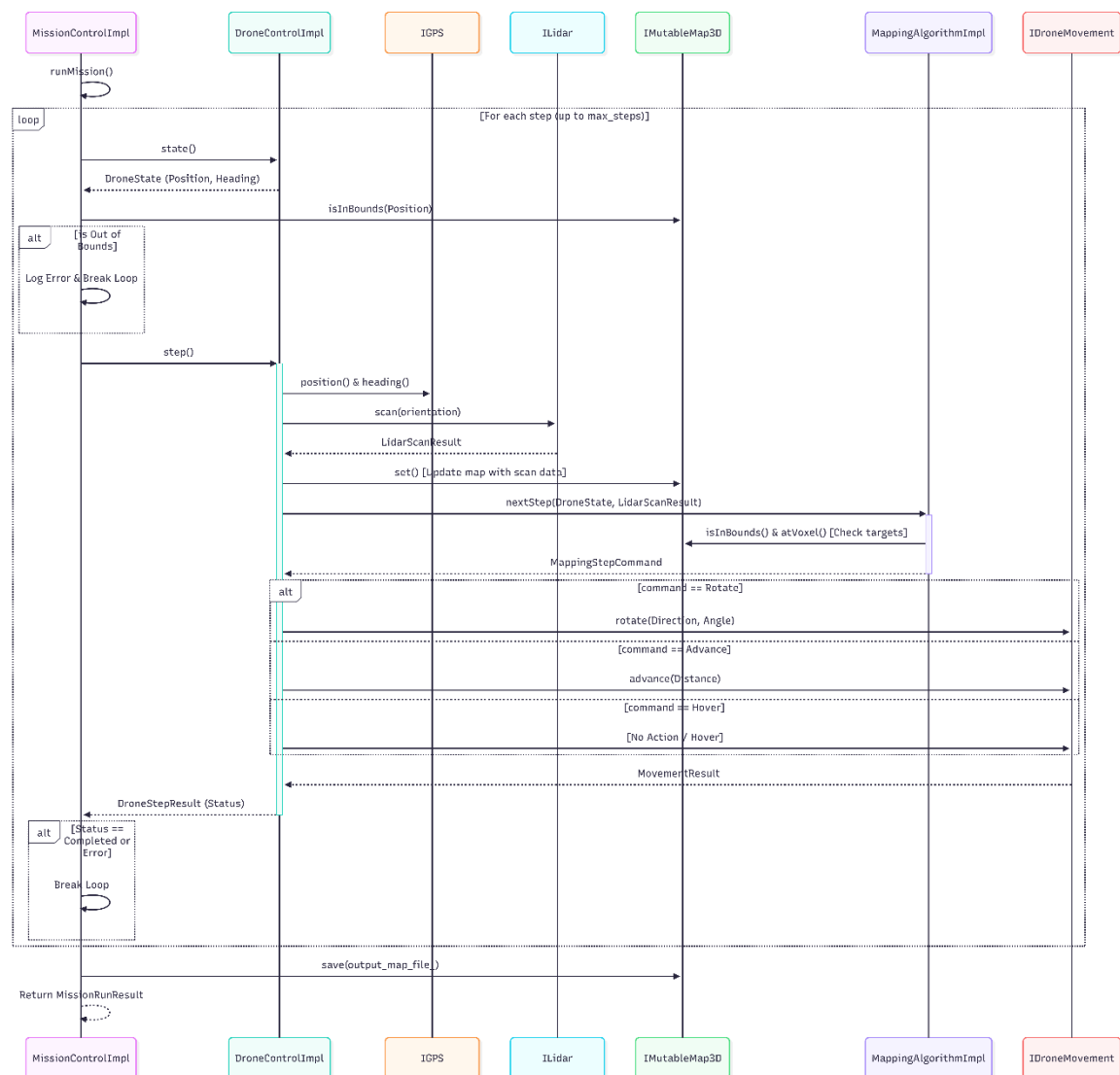
MissionControlImpl: Responsible for managing the mission's lifecycle and saving the final outputs.

DroneControlImpl: Acts as the executive "brain," mediating between the sensors (Lidar/GPS), actuators (DroneMovement), and the algorithm.

MappingAlgorithmImpl: Contains the algorithmic logic for mapping. It receives the current DroneState and returns movement commands (MappingStepCommand).

2. UML Sequence Diagram (Main Flow)

This diagram details the main execution flow within the runMission loop:



MissionControl starts the loop and calls step() on DroneControl.

DroneControl triggers nextStep in the algorithm.

The algorithm analyzes the map, calculates the next move, and returns a command.

DroneControl executes the movement via the Movement component and updates the map.

This process repeats until the mission is completed or an end signal is received.

3. Design Considerations and Alternatives

Managing Physical Units

Consideration: The project necessitates the use of the mp_units library.

Alternative: Using plain double types for all distance representations.

Decision: We adopted mp_units to manage physical units rigorously. We implemented a "safety layer" where numeric values are safely extracted (force_numerical_value_in(cm)) only at the endpoints of complex geometric calculations. This prevents unit conversion errors and ensures type-safe code while maintaining computational simplicity.

Dependency Injection

Consideration: The ability to execute tests without relying on the heavy simulator.

Decision: We utilized Interfaces for IGPS, ILidar, and IMap3D. This approach allowed us to inject Mock objects during Unit Tests, effectively isolating the algorithm's logic from external hardware or complex simulator dependencies

4. Testing Approach

Our testing strategy involved two main tiers:

Component Testing (Unit Tests): We employed Google Mock to simulate hardware interactions. We tested DroneControllImpl by defining "move and succeed" scenarios, verifying that the controller calls the algorithm and actuators exactly as expected.

Integration Testing: We executed the algorithm against actual map files (.npy) containing predefined obstacles. We utilized MapsComparison to ensure the mapping quality (Score) exceeded the 90% accuracy threshold.

Boundary Validation: We embedded strict boundary checks (isInBounds) within the algorithm and implemented an immediate error-logging mechanism (error_log.txt) for out-of-bounds attempts. This guaranteed system stability and prevented crashes (Segmentation Faults) when dealing with invalid mission configurations.